Artifical Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

- **Minimax Search**: A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally.
- **Alpha-Beta Pruning**: A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision.
- **Turing Test**: An operational test for intelligent behavior, the Imitation Game, where an interrogator tries to distinguish between a human and a machine based on their responses.
- **Game Tree**: A tree-like structure representing all possible sequences of moves in a game, with leaf nodes representing outcomes.
- **Perceptron**: A fundamental unit in neural networks that takes multiple inputs, applies weights to them, sums the weighted inputs, and produces an output based on a threshold.
- **Reinforcement Learning**: A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.
- **Matrix**: A rectangular array of numbers, symbols, or expressions arranged in rows and columns.
- **Linear Transform**: A function that maps vectors from one vector space to another, satisfying $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$ for all vectors u, v and scalars c.
- **Matrix Addition**: An element-wise operation where corresponding elements of two matrices of the same dimensions are added.
- **Scalar Multiplication**: An operation where each element of a matrix is multiplied by a scalar.
- **Matrix Inverse**: For a square matrix A, its inverse A^{-1} is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix.
- **Matrix Multiplication**: An operation where the product of an $m \times n$ matrix A and an $n \times p$ matrix B is an $m \times p$ matrix C, where $C_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$.
- **Matrix Transposition**: An operation that swaps the rows and columns of a matrix. The transpose of matrix A is denoted as A^T.
- **Derivative**: A measure of how a function changes with respect to a change in its input.
- **Partial Derivative**: A derivative of a function with multiple variables with respect to one of the variables, holding the others constant.
- **Gradient**: A vector of partial derivatives of a multivariable function.
- **Inductive Reasoning**: The process of deriving general principles from specific observations.

- **Linear Regression**: A machine learning algorithm used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.
- **Model**: A mathematical representation of a system or process, used to make predictions or understand relationships between variables.
- **Overfitting**: A phenomenon in machine learning where a model learns the training data too well, resulting in poor generalization to unseen data.
- **Binary Classification**: A machine learning task where the goal is to classify data into one of two categories.
- **Decision Boundary**: A line or surface that separates different classes of data in a classification problem.
- **Loss Function**: A function that quantifies the error of a model's predictions.
- **0-1 Loss**: A loss function that assigns a loss of 1 if the prediction is incorrect and 0 if it is correct.
- **Perceptron Loss**: A loss function used in the perceptron algorithm, defined as $\max\{0, -z\}$, where z = y(w\\$textasciicircum T$\x).
- **Hinge Loss**: A loss function used in support vector machines, defined as max0, 1 - z, where z = y(w\\$textasciicircum T$\x).
- **Logistic Loss**: A loss function used in logistic regression, defined as log(1 + e\\$textasciicircum -z$\), where z = y(w\\$textasciicircum T$\x).
- **Gradient Descent**: An iterative optimization algorithm used to find the minimum of a function by following the negative gradient.
- **Stochastic Gradient Descent**: A variant of gradient descent that updates the model's parameters using a randomly selected subset of the data.
- **Sigmoid Function**: A mathematical function that maps any real number to a value between 0 and 1, often used in logistic regression.
- **Bag of Features**: A representation of an image as a collection of its visual features (visual words), disregarding their spatial arrangement.
- **Image Classification**: The task of assigning a predefined label (e.g., "cat," "dog") to an input image.
- **Semantic Gap**: The difference between how humans understand images (semantically) and how computers represent them (numerically).
- **Viewpoint Variation**: The challenge in image classification caused by changes in the camera's or object's position and orientation.
- **Illumination**: The challenge in image classification caused by variations in lighting conditions.
- **Deformation**: The challenge in image classification caused by changes in the object's shape or pose.
- **Occlusion**: The challenge in image classification caused by parts of an object being hidden or obscured.
- **Background Clutter**: The challenge in image classification caused by distracting elements in the background.
- **Invariant Features**: Features that remain consistent across different variations of an object (e.g., different viewpoints, lighting, or poses).
- **Visual Words**: Basic visual elements extracted from images, used to represent the image's content in a Bag of Features model.

- **Category Models**: Representations of the frequency of visual words for a particular category in a Bag of Features model.
- **Nearest Neighbor (NN) Classifier**: A classification algorithm that assigns the label of the nearest training data point to each test data point.
- **Noise**: Random variations or unwanted disturbances in image data.
- **Salt and pepper noise**: Random occurrences of black and white pixels in an image.
- **Impulse noise**: Random occurrences of white pixels in an image.
- **Gaussian noise**: Variations in intensity drawn from a Gaussian normal distribution.
- **Mean Filtering**: A noise reduction technique that replaces each pixel with the average of its neighboring pixels.
- **Image Filtering**: The process of computing a function of the local neighborhood at each pixel in an image.
- **Blur Filter**: An image filter that smooths an image by averaging the values of neighboring pixels.
- **Identity Filter**: An image filter that leaves the image unchanged.
- **Sharpening Filter**: An image filter that enhances the edges and details in an image.
- **Textons**: Basic elements or patterns that characterize a texture.
- **Edge Detection**: The process of identifying points in a digital image where there is a significant change in intensity.
- **Intensity Profile**: A plot of intensity values along a single scanline (row or column) of an image.
- **Edges**: Curves in an image across which brightness changes significantly.
- **Corners/Junctions**: Points in an image where there are significant changes in intensity in multiple directions.
- **Corner Detection**: The process of identifying corners in an image, typically by looking for points where a small window's appearance changes significantly when translated.
- **Vision Pyramids**: Hierarchical representations of an image at multiple resolutions, often created by repeatedly blurring and subsampling the image.
- **Gaussian Pyramids**: A type of vision pyramid where the blurring operation is performed using a Gaussian filter.
- **Image Gradient**: A vector that points in the direction of the most rapid increase in intensity in an image. Its magnitude represents the strength of the edge.
- **Derivative of Gaussian Filter**: A filter created by taking the derivative of a Gaussian function. Used for edge detection.
- **Convolutional Neural Network (CNN) or ConvNet**: A type of artificial neural network designed to process data with a grid-like topology, such as images. It uses convolutional layers to extract features from the input data.
- **ImageNet Top-5 Classification Accuracy**: A metric used to evaluate the performance of image classification models on the ImageNet dataset. It represents the percentage of times the correct class is among the top 5 predictions made by the model.

- **AlexNet**: One of the first successful deep convolutional neural networks, known for its use of GPUs and its winning performance in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC).
- **Dense neural network**: A type of artificial neural network where each neuron in one layer is connected to every neuron in the next layer.
- **Convolution layer**: A layer in a convolutional neural network that applies a set of filters (kernels) to the input data to extract features. The filters are convolved with the input data to produce a feature map.
- **Pooling layer**: A layer in a convolutional neural network that reduces the spatial dimensions of the feature maps by applying a pooling function, such as max pooling or average pooling.
- **Multidimensional Convolution**: A convolution operation performed on input data with multiple channels (e.g., RGB channels in an image). Each channel is processed independently by a corresponding filter.
- **Local Receptive Field**: The small region of the input data that a neuron in a convolutional layer is connected to. This leads to sparse connectivity and reduces the number of parameters in the network.
- **Sparse Connectivity**: A characteristic of convolutional neural networks where each neuron is connected to only a few neurons in the previous layer, unlike fully connected networks where every neuron is connected to every other neuron.
- **Parameter Sharing**: A technique used in convolutional neural networks where the same set of weights (parameters) is used across different spatial locations in the input data. This significantly reduces the number of parameters in the network.
- **Rectified Linear Unit (ReLU)**: A non-linear activation function defined as $f(x) = \max(0, x)$. It outputs the input if it's positive and 0 otherwise.
- **Max Pooling**: A pooling operation that selects the maximum value within a defined region (e.g., 2x2) of the input feature map.
- **Average Pooling**: A pooling operation that calculates the average value within a defined region (e.g., 2x2) of the input feature map.
- **Back-propagation**: An algorithm used to train artificial neural networks by calculating the gradient of the loss function with respect to the network's weights and updating the weights accordingly.
- **LeNet-5**: A convolutional neural network architecture specifically designed for handwritten digit recognition on the MNIST dataset.
- **MNIST dataset**: A large database of handwritten digits commonly used for training and testing in the field of machine learning.
- **CIFAR-10 dataset**: A dataset of 60,000 32x32 color images classified into 10 classes, used for image classification tasks.
- **Held Out Data**: A subset of the training data used to tune hyperparameters of a model.
- **Test Data**: A subset of the data used to evaluate the performance of a trained model on unseen data.
- **Training Data**: The subset of data used to train a machine learning model.
- **Regularization Coefficient**: A hyperparameter (λ) that controls the strength of regularization in a model, balancing training error and model complexity.
- **Regularizer**: A penalty term added to the loss function to prevent overfitting by discouraging large weights.

- **LASSO**: A type of regularization that uses the L1 norm as the regularizer.
- **Learning Curves**: Graphs showing the relationship between model complexity and accuracy on training and test data, used to diagnose underfitting and overfitting.
- **S-fold Cross Validation**: A model evaluation technique where the data is split into S folds, each used once as a validation set while the others are used for training.
- **Matrix Factorization**: A technique that decomposes a high-dimensional data matrix into a product of two lower-dimensional matrices, revealing latent factors that capture underlying structure in the data.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction technique that identifies the principal components (directions of maximum variance) in a dataset to reduce dimensionality while preserving most of the data's variance.
- **Dimensionality Reduction**: Methods that transform data from a higher number of dimensions to a lower number of dimensions, often to discard irrelevant features, visualize high-dimensional data, or reflect the intrinsic dimensionality of the data.
- **Low Rank Matrix Approximations**: Approximating a high-dimensional matrix with a lower-rank matrix, effectively reducing the dimensionality of the data while preserving essential information.
- **Embeddings**: A low-dimensional representation of data points, often obtained through dimensionality reduction techniques like matrix factorization, that captures the essential relationships between data points.
- **Collaborative Filtering**: A recommender system technique that predicts a user's rating for an item based on the ratings of similar users.
- **Recommender Systems**: Systems that predict user preferences for items (e.g., movies, products) based on past user behavior and item characteristics.
- **EigenSNP**: Eigenvectors obtained from Principal Component Analysis (PCA) applied to a matrix of Single Nucleotide Polymorphisms (SNPs). They represent directions of maximum variance in the genetic data, capturing population structure.
- **Single Nucleotide Polymorphisms (SNPs)**: The most common type of genetic variation in the genome, representing locations where two alternate nucleotide bases are observed.
- **Bias-variance tradeoff**: The balance between model complexity and prediction accuracy. High complexity models can overfit the training data (high variance), while low complexity models may underfit (high bias).
- **Clustering**: The process of grouping a set of objects into classes of similar objects. Objects within a cluster should be similar, and objects from different clusters should be dissimilar.
- **K-means clustering**: A commonly used clustering algorithm that partitions objects into K disjoint subsets, assuming the objects are p-dimensional vectors and using a distance measure (typically Euclidean distance) between them.
- **Euclidean distance**: The most commonly used distance measure for continuous data, defined as $d_E(x, y) = \sqrt{\sum_{i=1}^{p}(x_i - y_i)^2}$, where p is the number of attributes/dimensions.
- **Hamming distance**: A distance measure used for bit/binary vectors that counts the number of bits that differ. It can be generalized to discrete data to count mismatches.
- **Matching coefficient**: A normalized Hamming distance, obtained by dividing the Hamming distance by the number of attributes/bits.

- **Inertia**: In the context of K-means, the sum of squared distances from each point to its cluster centroid. Minimizing inertia is the goal of K-means optimization.
- **Purity**: A measure of the extent to which clusters contain objects of a particular class. It's calculated as $\text{purity}(C, G) = \frac{1}{N}\sum_{i=1}^{K} \max_j N_j \in G_i$, where N is the total number of data points, K is the number of clusters, and N_j is the number of examples of class j in cluster i.
- **Personas**: Representations of different user types or target audiences, used to provide a common understanding across teams.
- **Intelligent Agent**: A system that perceives its environment and takes actions that maximize its chances of success.
- **Logic Reasoning**: A cornerstone of Artificial Intelligence; the process of using rules of deduction to derive a conclusion from given assumptions.
- **Satisfiability Problem (SAT)**: The problem of determining whether a given propositional formula is satisfiable, i.e., whether there exists an assignment of truth values to its variables that makes the formula true.
- **3-Satisfiability (3-SAT)**: A restricted version of the satisfiability problem where each clause in the propositional formula contains exactly three literals.
- **Propositional Logic**: The simplest form of logic, where sentences are represented by propositional symbols and combined using logical connectives such as negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), implication ($\implies$), and biconditional ($\iff$).
- **Literal**: A propositional variable or its negation.
- **Clause**: A disjunction of literals.
- **Conjunctive Normal Form (CNF)**: A propositional formula that is a conjunction of clauses.
- **Logical Equivalence**: A relationship between two logical expressions where they have the same truth value under all possible assignments of truth values to their variables.
- **Equisatisfiable**: Two formulas are equisatisfiable if they are both satisfiable or both unsatisfiable.
- **Classical Planning**: A type of planning problem where the goal is to find a sequence of actions that achieves a desired state from an initial state.
- **Combinatorial Design**: The study of mathematical structures with desirable properties, such as Latin squares, quasi-groups, Ramsey numbers, and Steiner systems.
- **Recursion**: A process where a function calls itself, eventually terminating in a base case.
- **Base Case**: The simplest form of a problem in a recursive approach, providing a direct solution without further recursion.
- **Recursive Case**: In a recursive approach, the same problem/method/data structure, but smaller and closer to the base case.
- **Breadth First**: A tree traversal method that visits nodes level by level.
- **Depth First**: A tree traversal method that traverses down a branch of a tree.
- **Structural Recursion**: A recursive approach applied to data structures like linked lists, trees, or graphs.
- **Problem Solving as Search**: An approach to problem-solving that involves exploring possibilities to find a solution.

- **State**: A description of the configuration of an environment in a search problem.
- **Action**: Activities that move from one state to another in a search problem.
- **Search Algorithm**: Determines which actions should be tried in which order to find a solution in a search problem.
- **Redundant States**: States that are explored multiple times in a search, leading to inefficiency.
- **Dynamic Programming**: An optimization technique that avoids redundant calculations by storing and reusing previously computed values.
- **Graph**: A data structure consisting of nodes and edges, representing relationships between entities.
- **Successor Function**: A function that maps a state to its possible successor states in a search problem.
- **World State**: A complete description of every detail of the environment in a search problem.
- **Search State**: A simplified representation of the environment, containing only the details needed for planning in a search problem.
- **Rational Agent**: An agent that acts to maximize its goal achievement, given available information.
- **SAT Solving**: The process of finding a satisfying assignment for a given Boolean formula in CNF, or determining that no such assignment exists.
- **Hamiltonian Path Problem**: Determining if a path exists in a graph that visits every vertex exactly once.
- **Graph Coloring Problem**: Assigning colors to the vertices of a graph such that no two adjacent vertices have the same color.
- **Unit Clause**: A clause in a CNF formula containing only one literal.
- **Unit Propagation**: A technique in SAT solving that efficiently assigns truth values to variables based on unit clauses.
- **Pure Literal**: A literal in a CNF formula that appears only positively or only negatively.
- **DPLL Algorithm**: A complete and backtracking-based algorithm for solving the SAT problem.
- **Resolution Rule**: An inference rule in propositional logic used to derive new clauses from existing ones.
- **Clause Learning**: A technique in SAT solving where clauses are learned from conflicts encountered during the search process to avoid similar conflicts in the future.
- **1-UIP Scheme**: A specific scheme for selecting the learned clause in clause learning.
- **Implication Graph**: A graph representing the implications between variables in a SAT problem.
- **Automated Reasoning**: The field of computer science focused on developing systems that can reason and solve problems automatically.
- **Model Generator (Encoder)**: A component in an automated reasoning system that transforms a problem instance into a formal model suitable for inference.
- **General Inference Engine**: A component in an automated reasoning system that performs inference on the model generated by the model generator.
- **Bias**: A systematic error in a dataset or algorithm that leads to unfair or inaccurate outcomes for certain groups.

- **Fairness**: The property of an algorithm or system to treat all individuals or groups equitably, without discrimination.
- **Disparate Treatment**: A type of discrimination where decisions are (partly) based on a subject's protected attribute information.
- **Disparate Impact**: A type of discrimination where outcomes disproportionately hurt (or benefit) people with certain sensitive attribute values.
- **Proxies for Discrimination**: Seemingly neutral data points that can act as stand-ins for discriminatory factors, perpetuating bias in algorithms.
- **Anonymized Data**: Data that has had identifying information removed to protect privacy. However, it is not always truly safe from re-identification.
- **Interpretable Results**: Results from an AI algorithm that are easily understandable and explainable, increasing trust and transparency.
- **Group Fairness**: A fairness criterion that requires equalizing outcomes for different groups.
- **Individual Fairness**: A fairness criterion that requires treating similar individuals similarly.

2 Problem Types

- **Understanding the Nature of Intelligence**: This problem explores the definition and characteristics of intelligence, both human and artificial. It examines various perspectives on what constitutes intelligence and how it can be measured or simulated.
- **Defining Artificial Intelligence**: This problem focuses on establishing a precise definition of artificial intelligence, considering different perspectives and historical contexts. It examines the evolution of AI definitions and their implications.
- **Passing the Turing Test**: This problem investigates the implications of passing the Turing Test as a measure of artificial intelligence. It explores whether passing the test truly indicates intelligence or merely the ability to mimic human behavior.
- **Search Space Complexity in Game Playing**: This problem addresses the computational challenges posed by the vast search spaces in complex games like checkers, chess, and Go. It explores the limitations of brute-force search and the need for more efficient algorithms.
- **Overcoming Limitations of Search Algorithms**: This problem focuses on the limitations of search-based approaches to AI and the need for incorporating machine learning techniques to handle the complexity of real-world problems.
- **Addressing the Challenges of Computer Vision**: This problem examines the historical context and ongoing challenges in computer vision, highlighting the gap between initial expectations and the persistent difficulties in achieving human-level performance.
- **Vulnerability of Neural Networks to Adversarial Attacks**: This problem explores the susceptibility of neural networks to adversarial attacks, where small, carefully crafted perturbations can lead to misclassifications.
- **Ethical Considerations in AI**: This problem addresses the ethical implications of AI systems, particularly concerning bias and fairness. It examines the potential for AI to perpetuate or amplify existing societal biases.
- **Differences Between Game Playing Paradigms**: This problem explores the differences between game playing in structured environments like chess and more dynamic, unpredictable environments like soccer, highlighting the challenges posed by each.

- **Similarities Between Seemingly Disparate AI Tasks**: This problem investigates the commonalities between different AI tasks, such as speech synthesis and game playing, to identify underlying principles and shared algorithmic components.
- **The AI Grand Challenge of Autonomous Vehicles**: This problem examines the reasons behind the selection of autonomous vehicles as a major challenge in AI research, considering the complexity of the task and its societal impact.
- **Understanding the AlphaGo System**: This problem explores the architecture and functionality of the AlphaGo system, highlighting its combination of deep learning and search algorithms.
- **Representing Linear Transformations with Matrices**: Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations. This is fundamental to many AI algorithms.
- **Matrix Arithmetic**: The slide covers basic matrix operations such as addition, subtraction, scalar multiplication, and multiplication, which are essential for many AI computations.
- **Solving Systems of Linear Equations using Matrices**: Systems of linear equations can be represented and solved efficiently using matrix operations, particularly using matrix inverses.
- **Matrix Transposition**: Matrix transposition is a fundamental operation that involves swapping rows and columns of a matrix, useful for data manipulation in AI.
- **Understanding Perceptrons**: A perceptron, a fundamental building block of neural networks, is introduced, illustrating its structure and function through a diagram.
- **Parallelizing Matrix Multiplication**: Domain decomposition is presented as a method for parallelizing matrix multiplication, improving computational efficiency in AI.
- **Calculating Derivatives**: The slide shows the derivation of the derivative of the sigmoid function, a crucial step in training neural networks.
- **Linear Regression**: Finding the line that best fits a set of data points, minimizing the sum of squared distances between the points and the line.
- **Multiple Linear Regression**: Extending linear regression to include multiple independent variables to predict a dependent variable.
- **Curve Fitting**: Fitting a curve to a set of data points, potentially using polynomial functions of higher degrees.
- **Overfitting**: The phenomenon where a model fits the training data too closely, resulting in poor generalization to unseen data.
- **Binary Classification**: Classifying data points into two distinct categories using a learned function.
- **Choosing a Loss Function**: Selecting an appropriate loss function (e.g., 0-1 loss, perceptron loss, hinge loss, logistic loss) to measure the error of a classification model.
- **Gradient Descent**: An iterative optimization algorithm used to minimize the loss function by following the direction of the negative gradient.
- **Stochastic Gradient Descent**: A variant of gradient descent that updates the model parameters using a randomly selected subset of the data points at each iteration.
- **Perceptron Algorithm**: A simple algorithm for binary classification that iteratively updates the weight vector based on misclassified data points.
- **Logistic Regression**: A probabilistic model for binary classification that uses the sigmoid function to map the linear combination of input features to a probability.

- **Image Classification**: The problem of assigning a label from a predefined set to an input image. Examples include classifying images of cats, dogs, trucks, and planes.
- **Addressing the Semantic Gap in Image Classification**: The challenge of bridging the gap between a computer's numerical representation of an image and a human's semantic understanding of it.
- **Handling Viewpoint Variation in Image Classification**: The difficulty of classifying images of the same object from different viewpoints.
- **Addressing Illumination Variation in Image Classification**: The challenge of classifying images of the same object under different lighting conditions.
- **Handling Deformation in Image Classification**: The difficulty of classifying images of the same object in different poses or shapes.
- **Addressing Occlusion in Image Classification**: The challenge of classifying images where parts of the object are hidden or obscured.
- **Handling Background Clutter in Image Classification**: The difficulty of classifying images where the object of interest is surrounded by distracting elements.
- **Finding Invariant Features for Image Classification**: The need to identify features that remain consistent across different variations of the same object.
- **Object Classification using Bag of Features**: A method for image classification that represents images as collections of visual words (features).
- **Texture Recognition**: The problem of identifying and classifying textures based on the repetition of basic elements or textons.
- **Noise Reduction in Images**: The problem of removing or reducing unwanted variations in pixel intensities in images.
- **Image Filtering**: The process of applying a filter (or mask) to an image to modify pixel values based on their local neighborhood.
- **Image Sharpening**: A technique to enhance the details and edges in an image by reversing the effects of blurring.
- **Document Retrieval**: The problem of finding relevant documents based on their content, often represented by word frequencies.
- **Bag of Features Object Categorization**: This problem involves using Bag of Features to categorize objects by mapping their feature vectors into a model space.
- **Edge Detection**: This problem focuses on identifying edges in images, which are curves where brightness changes significantly.
- **Corner Detection**: This problem involves identifying corners in images, which are points where the image's intensity changes significantly in multiple directions.
- **Vision Pyramids**: This problem involves representing an image as a "pyramid" of progressively smaller and blurred versions to extract features at different scales.
- **Object Detection and Recognition using CNNs**: Convolutional Neural Networks are used to identify and locate objects within images and videos. Examples include facial feature detection, car detection in highway scenes, and fruit recognition.
- **Improving ImageNet Classification Accuracy**: This problem focuses on improving the accuracy of image classification models over time, as demonstrated by the decreasing error rate in the ImageNet challenge. AlexNet is highlighted as a significant milestone.

- **Comparing Dense and Convolutional Neural Networks**: This problem involves understanding the architectural differences between standard fully connected neural networks and convolutional neural networks, particularly concerning their connectivity patterns and data processing in multiple dimensions.
- **Understanding Artificial Neurons**: This problem focuses on understanding the fundamental operations of an artificial neuron, including weighted input summation and the application of activation functions to introduce non-linearity.
- **Convolution and Pooling Operations in CNNs**: This problem involves understanding the mathematical operations of convolution and pooling layers in CNNs, including the application of convolution filters and the downsampling effect of pooling.
- **Explaining Sparse Connectivity in Convolutional Layers**: This problem explores the reasons behind using convolutional layers instead of fully connected layers, focusing on the concept of sparse connectivity and parameter sharing.
- **Understanding the ReLU Activation Function**: This problem involves understanding the mathematical definition and graphical representation of the Rectified Linear Unit (ReLU) activation function.
- **Understanding Pooling Layers**: This problem focuses on understanding the downsampling operation performed by pooling layers in CNNs, including max pooling and average pooling.
- **Backpropagation for Neural Network Training**: This problem involves understanding the backpropagation algorithm used to train neural networks by adjusting weights to minimize the error between predicted and desired outputs.
- **Understanding a Simple CNN Structure**: This problem involves understanding the architecture of a simple CNN, including the sequence of convolutional, activation, pooling, and fully connected layers.
- **Visualizing Learned Convolution Filters**: This problem involves understanding the learned filters in a CNN and visualizing their patterns and textures.
- **Multidimensional Convolution**: This problem involves understanding how convolution operates on multi-channel images, such as RGB images.
- **Biological Inspiration for Artificial Neural Networks**: This problem involves understanding the biological inspiration behind artificial neural networks, drawing parallels between biological neurons and perceptrons.
- **Using the MNIST Dataset for Handwritten Digit Recognition**: This problem involves understanding the MNIST dataset and its use in training and testing machine learning models for handwritten digit recognition.
- **Applying LeNet-5 to the MNIST Dataset**: This problem involves understanding the LeNet-5 architecture and its application to the MNIST dataset, including the dimensions and operations of each layer.
- **Understanding the CIFAR-10 Dataset and State-of-the-Art Models**: This problem involves understanding the CIFAR-10 dataset and comparing the performance of different state-of-the-art models on this dataset.
- **Generalization and Overfitting**: This problem focuses on the ability of a model to generalize to unseen data, contrasting well-generalized models with overfitted models that perform poorly on new data.
- **Training \textbackslash{}& Testing Errors**: This problem involves analyzing the prediction errors on training and testing datasets to diagnose issues like overfitting and underfitting, and to understand the bias-variance tradeoff.
- **Training and Testing**: This problem addresses the process of splitting data into training, held-out (validation), and testing sets for model development and evaluation.

- **Hyper-Parameter Tuning**: This problem focuses on finding the optimal settings for hyperparameters (parameters that control the learning process, not learned from data) to improve model performance.
- **Fit a cubic function with a degree-9 polynomial**: This problem illustrates overfitting by fitting a high-degree polynomial to data, resulting in a model that fits the training data perfectly but generalizes poorly.
- **Control the Model Complexity**: This problem explores methods for controlling model complexity to prevent overfitting, such as using regularization techniques or limiting the degree of a polynomial.
- **Tuning on Held-Out Validation Data**: This problem addresses the use of a held-out validation set to tune hyperparameters, preventing overfitting to the training data.
- **Learning curves illuminate likely causes of error**: This problem involves interpreting learning curves (plots of model performance vs. training data size or model complexity) to diagnose underfitting or overfitting.
- **When to get more data vs. use different model**: This problem focuses on deciding whether to gather more data or change the model based on the learning curves and the presence of underfitting or overfitting.
- **S-fold cross validation**: This problem describes the process of using S-fold cross-validation to evaluate model performance and select hyperparameters robustly.
- **The Data Analysis Pipeline**: This problem outlines the stages of a data analysis pipeline, highlighting potential issues at each stage, from data collection to model application.
- **Matrix Factorization**: Approximating a high-dimensional data matrix as a product of two lower-dimensional matrices to reveal latent factors.
- **Dimensionality Reduction**: Transforming data from a higher number of dimensions to a lower number of dimensions to discard irrelevant features, visualize high-dimensional data, or reflect the intrinsic dimensionality.
- **Low Rank Matrix Approximations**: Reducing the rank of a matrix to achieve dimensionality reduction, using equivalent definitions based on linearly independent subsets of columns/rows or matrix factorization.
- **Singular Value Decomposition (SVD)**: Decomposing a matrix into three simpler matrices (U, S, V) to reveal latent factor affinities for entities and attributes.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data.
- **Recommender System Algorithms**: Inferring missing ratings in a user-item matrix based on the ratings of similar users.
- **Collaborative Filtering**: Predicting a user's rating for an item based on the ratings of similar users.
- **Modeling Systematic Biases**: Accounting for biases in user ratings and movie ratings to improve the accuracy of recommender systems.
- **Bias-Variance Tradeoff**: Balancing model complexity (number of parameters) and prediction accuracy (RMSE) to optimize the performance of a recommender system.
- **Clustering Data**: Grouping a set of objects into classes of similar objects, where objects within a cluster are similar and objects from different clusters are dissimilar. This is a common unsupervised learning task with applications in various fields.

- **Supervised vs. Unsupervised Learning**: Distinguishing between supervised learning (learning from paired inputs/outputs) and unsupervised learning (learning from inputs only). Examples of each are given, including classification and regression for supervised learning, and clustering and matrix factorization for unsupervised learning.
- **Defining Clustering**: Describing clustering as the process of grouping objects into classes of similar objects, with the goal of maximizing intra-cluster similarity and minimizing inter-cluster similarity.
- **Choosing the Number of Clusters (k) in K-means**: Determining the optimal number of clusters for the K-means algorithm, balancing accuracy (low inertia) and complexity (small k). The "elbow method" is presented as a visual approach to selecting k.
- **Evaluating Clustering Results**: Assessing the quality of clustering results, both subjectively (visual inspection of proximity matrices and PCA-reduced visualizations) and objectively (using metrics like purity and similarity matrices when class labels are available).
- **K-means Clustering Algorithm**: Describing the iterative process of the K-means algorithm, including initialization, data assignment, and relocation of means.
- **Measuring Distance Between Data Points**: Using distance measures like Euclidean distance for continuous data and Hamming distance/matching coefficient for discrete data to quantify the dissimilarity between data points.
- **Defining Reasoning Using Logic**: Reasoning is defined as proving a conclusion from given assumptions using rules of deduction. Examples include proving mathematical theorems or determining winning moves in games.
- **The Satisfiability Problem (SAT)**: Determining whether a given propositional formula is satisfiable, meaning there exists an assignment of truth values to its variables that makes the formula true.
- **Converting Logical Expressions to Conjunctive Normal Form (CNF)**: Transforming arbitrary logical expressions into an equivalent CNF representation, which is a conjunction of clauses, each being a disjunction of literals.
- **Applications of SAT Solvers**: Using SAT solvers to address problems in various domains, including model checking, classical planning, combinatorial design, test pattern generation, scheduling, optimal control, protocol design, multi-agent systems, and e-commerce.
- **Reducing SAT to 3-SAT**: Transforming a general SAT problem into an equivalent 3-SAT problem, where each clause contains exactly three literals. This is achieved by introducing new variables and creating a conjunction of clauses that are equisatisfiable with the original clause.
- **Representing Recursion**: This problem focuses on understanding and implementing recursive algorithms and data structures, such as the Fibonacci sequence and tree traversal.
- **Problem Solving as Search**: This problem involves formulating problems as search problems, defining states, actions, and goal tests, and choosing appropriate search algorithms.
- **Optimizing Search Algorithms**: This problem addresses the efficiency of search algorithms, particularly in the context of redundant states and computational complexity.
- **Representing Problems as Graphs**: This problem involves representing problems, such as pathfinding and the 8-puzzle, as graphs and applying graph search algorithms.
- **Solving the Satisfiability Problem (SAT)**: Determining whether a given Boolean formula is satisfiable, meaning there exists an assignment of truth values to its variables that makes the formula evaluate to true.
- **General Automated Reasoning**: Developing better reasoning and modeling technology to solve problems across various domains, such as logistics, chess, planning, and verification.

- **Addressing Exponential Complexity Growth**: Tackling the challenge of exponential complexity growth in problem-solving, particularly in propositional reasoning, where problem size dramatically increases computational difficulty.
- **Encoding Real-World Problems into SAT**: Transforming real-world problems, such as model checking, planning, and combinatorial design, into SAT instances to leverage the efficiency of SAT solvers.
- **Addressing Bias and Fairness in Data**: This problem focuses on identifying and mitigating biases in data used to train AI models, which can lead to unfair or discriminatory outcomes. Examples include bias in loan applications, hiring processes, and criminal justice risk assessments.
- **Ensuring Privacy in Data Analysis**: This problem addresses the challenges of protecting individual privacy while utilizing data for AI applications. Examples include anonymization techniques and the potential for re-identification of individuals from seemingly anonymized datasets.
- **Understanding and Mitigating Malicious Attacks on AI Systems**: This problem explores the vulnerabilities of AI systems to intentional attacks aimed at subverting their functionality or causing harm.
- **Defining and Achieving Fairness in Algorithmic Decision-Making**: This problem investigates different definitions of fairness (e.g., group fairness, individual fairness) and explores methods for ensuring fair outcomes from AI algorithms.
- **Improving the Interpretability and Trustworthiness of AI Models**: This problem focuses on developing techniques to make the decision-making processes of AI models more transparent and understandable, thereby increasing trust in their outputs.

3 Algorithm Solutions

- **Minimax Search**: A recursive algorithm for two-player zero-sum games that explores the game tree to find the best move for the current player, assuming the opponent also plays optimally. It alternates between maximizing the score for the current player and minimizing the score for the opponent.
- **Alpha-Beta Pruning**: An optimization technique for Minimax search that reduces the search space by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha and beta values to prune branches.
- **Matrix Addition**: $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication**: $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$
- **Matrix Multiplication**: $C_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$
- **Matrix Transposition**: Swapping rows and columns of a matrix A to obtain A^T.
- **Solving Linear Equations with Invertible Matrices**: $Ax = b \implies x = A^{-1}b$
- **Derivative of Sigmoid Function**: $\frac{d}{dx}\sigma(x) = \frac{e^{-x}}{(1+e^{-x})^2}$
- **Multiple Linear Regression**: Find the optimal coefficients β that minimize the sum of squared errors between the observed and predicted values. This is done by solving the equation $\beta^{=}(X^T X)^{-1} X^T y$, where X is the design matrix, y is the vector of observed values, and β is the vector of coefficients.
- **Perceptron Algorithm**: Iteratively update the weight vector w by adding $y_n x_n$ if the data point x_n is misclassified, where y_n is the true label and x_n is the feature vector. If the data is linearly separable, this algorithm will find a separating hyperplane.

- **Logistic Regression**: Minimize the logistic loss function $F(w) = \sum_{i=1}^{N} \log(1 + e^{-y_i w^T x_i})$ using stochastic gradient descent. The weight vector w is updated iteratively using the rule $w \leftarrow w + \alpha(\sigma(-y_n w^T x_n) y_n x_n)$, where α is the learning rate and σ is the sigmoid function.
- **Bag of Features**: Represent images by frequencies of "visual words" (Bags of features)
- **Nearest Neighbor Classifier**: Assign label of nearest training data point to each test data point
- **Moving Average**: Replace each pixel with an average of all the values in its neighborhood
- **Weighted Moving Average**: Add weights to our moving average
- **Image Sharpening**: $f_{sharp} = f + \alpha(f f_{blur})$
- **Finite Difference**: $\frac{df}{dx}[x, y] \approx F[x + 1, y] - F[x, y]$
- **Image Gradient Magnitude**: $||\nabla f|| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$
- **Image Gradient Direction**: $\theta = \tan^{-1}(\frac{\partial f}{\partial x} / \frac{\partial f}{\partial y})$
- **Sum of Squared Differences (SSD)**: $E(u, v) = \sum_{(x,y) \in W} [I(x + u, y + v) - I(x, y)]^2$
- **Anisotropic Gaussian**: $G_{\sigma_x, \sigma_y}(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \cdot e^{-\frac{1}{2}((\frac{x^2}{\sigma_x^2}) + (\frac{y^2}{\sigma_y^2}))}$
- **Isotropic Gaussian**: $G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \cdot e^{-\frac{r^2}{2\sigma^2}}$
- **Convolution**: A convolution operation involves sliding a filter (kernel) across an input image, performing element-wise multiplication between the filter and the corresponding input region, and summing the results to produce a single output value. This process is repeated for every possible position of the filter on the input image, resulting in an output feature map.
- **Max Pooling**: This operation involves dividing the input feature map into non-overlapping regions (e.g., 2x2) and selecting the maximum value within each region as the output. This reduces the spatial dimensions of the feature map while retaining the most prominent features.
- **Average Pooling**: This operation involves dividing the input feature map into non-overlapping regions (e.g., 2x2) and calculating the average value within each region as the output. This reduces the spatial dimensions of the feature map while retaining a summary of the features.
- **Backpropagation**: This algorithm is used to train neural networks by calculating the gradient of the loss function with respect to the network's weights. The gradient is then used to update the weights iteratively, minimizing the loss and improving the network's performance.
- **ReLU Activation Function**: This function outputs the input if it is positive, and 0 otherwise. Mathematically, $f(x) = \max(0, x)$. This introduces non-linearity into the network, enabling it to learn complex patterns.
- **Regularized Linear Regression**: $E(w) = RSS(w) + \lambda R(w)$, where $R(w)$ is either $||w||_2^2$ or $||w||_1$, and λ controls the trade-off between training error and complexity.
- **S-fold Cross Validation**: Split training data into S equal parts; use each part as a validation set, training on the others; choose hyperparameters based on average performance.

- **Singular Value Decomposition (SVD)**: $A = USV^T$, where A is the input matrix, U and V are orthogonal matrices, and S is a diagonal matrix containing singular values.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data. The eigenvectors of the covariance matrix correspond to the principal components.
- **Matrix Factorization for Recommender Systems**: Approximates a user-item rating matrix as a product of two lower-dimensional matrices representing user and item latent factors. $r_{i,j} \approx u_i \cdot v_j$, where $r_{i,j}$ is the rating of user i for item j, u_i is the user's latent factor vector, and v_j is the item's latent factor vector.
- **Modeling Systematic Biases in Recommender Systems**: $r_{ij} \approx \mu + b_i + b_j + u_i \cdot v_j$, where r_{ij} is the rating, μ is the overall mean rating, b_i is the user bias, b_j is the item bias, and $u_i \cdot v_j$ represents user-item interactions.
- **Root Mean Square Error (RMSE)**: $\frac{1}{|R|} \sum_{(u,i) \in R} (r_{ui} - \hat{r}_u)^2$, where R is the set of ratings, r_{ui} is the actual rating, and $\hat{r}_u$ is the predicted rating.
- **Euclidean Distance**: $d_E(x, y) = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2}$, where p is the number of attributes/dimensions.
- **Hamming Distance**: Counts the number of bits that differ between two bit/binary vectors.
- **Matching Coefficient**: Hamming distance normalized by the number of attributes/bits.
- **Purity**: $\text{purity}(C, G) = \frac{1}{N} \sum_{i=1}^{K} \max_j N_j \in G_i$, where N is the total number of data points, K is the number of clusters, and N_j is the number of examples of class j in cluster i.
- **Conversion to CNF**: This algorithm transforms a logical expression into Conjunctive Normal Form (CNF) by applying logical equivalences to eliminate implications, biconditionals, and negations, and then using distributive laws to flatten the expression into a conjunction of clauses.
- **SAT Problem**: This problem determines if a given propositional formula (often in CNF) is satisfiable, meaning there exists an assignment of truth values to its variables that makes the formula true.
- **3-SAT Reduction**: This algorithm reduces an unrestricted SAT problem to a 3-SAT problem by transforming each clause with more than three literals into a conjunction of clauses with exactly three literals using auxiliary variables. The resulting formula is equisatisfiable with the original.
- **Recursive Problem Solving**: Do part of the work and reduce the rest into a similar but smaller problem.
- **Structural Recursion**: Traversal of data structures like trees using recursion. Breadth-First Search visits nodes level by level; Depth-First Search traverses down a branch.
- **Structural Recursion (Binary Tree Example)**: Searching a binary search tree recursively. If the current node's value (V) equals the target value (X), return True. If there are no children, return False. If $X < V$, search the left subtree; if $X > V$, search the right subtree.
- **Method and Structural Recursion**: Combining method and structural recursion to search a tree structure that is generated as the search progresses.
- **Searching States (Chess)**: Creating and modeling states of a chessboard to evaluate moves and choose the best sequence of actions. The search tree is large, and searching the entire space is infeasible.

- **Mazes as a Search Problem**: Representing a maze as a search problem by defining states (locations), actions (moves), and a search algorithm to find a path from the start to the goal.
- **Optimizing Fibonacci Recursion**: Removing redundant calculations by storing and reusing previously computed values (dynamic programming or memoization).
- **Searching Graphs and Maps**: Extending search algorithms from trees to graphs, handling multiple paths to the same state and evaluating which path is best.
- **8-Puzzle as a Graph**: Representing the 8-puzzle as a graph, where states are tile configurations and actions are tile movements. The search space contains loops, and the first solution found may not be optimal.
- **Traveling Salesperson Problem as a Graph**: Representing the traveling salesperson problem as a graph, where states are cities and actions are movements between cities. The goal is to find the shortest path visiting all cities.
- **Robotic Assembly as a Search Problem**: Representing robotic assembly as a search problem, where states are joint angles and part positions, actions are joint movements, and the goal is complete assembly.
- **Assumptions of Simple Search**: Static (world doesn't change), Discrete (finite states), Observable (states are known), Deterministic (actions have certain outcomes).
- **Pac-Man as an Agent**: Modeling Pac-Man as an agent interacting with an environment, receiving percepts through sensors and performing actions through actuators.
- **State Space Sizes**: Calculating the size of the state space for different problems, highlighting the combinatorial explosion in complex environments.
- **Naïve\textbackslash{}\textbackslash{}_SAT**: A recursive algorithm that explores all possible variable assignments to determine the satisfiability of a propositional formula. It checks for immediate satisfiability or unsatisfiability, then recursively explores assignments for an unassigned variable, backtracking if necessary.
- **DPLL**: A recursive algorithm that improves upon the Naïve\\$_S ATalgorithmbyincorporatingBooleanConstraintPropaga$ $Asub-algorithmwithinDPLLthatrepeatedlyidentifiesandassignsvaluestoliteralsinunitclauses(clauseswithonlyoneun$

- **Resolution Rule**: An inference rule that derives a new clause from two clauses containing a literal and its negation, by combining the remaining literals with a logical OR.
- **Clause Learning**: A technique used to learn from conflicts encountered during the DPLL search. When a conflict occurs, a clause is learned that prevents the algorithm from repeating the same mistake in the future.
- **Hamiltonian Path Encoding**: A method to encode the Hamiltonian Path Problem into a SAT problem. This involves creating Boolean variables representing whether a vertex is visited at a specific position in the path and using clauses to enforce constraints such as the path using only existing edges and visiting each vertex exactly once.
- **Graph Coloring Encoding**: A method to encode the Graph Coloring Problem into a SAT problem. This involves creating Boolean variables representing whether a vertex is assigned a specific color and using clauses to enforce constraints such as adjacent vertices having different colors.
  - **Fairness through Blindness**: Ignore protected attributes during model training and prediction.
  - **Group Fairness**: $P(\text{outcome } o|S) = P(\text{outcome } o|T)$, where S and T are two groups. This is not a strong enough notion of fairness.
  - **Individual Fairness**: Treat similar individuals similarly; similar individuals should be treated similarly for the purpose of the classification task and similarly distributed over outcomes.
  - **Detecting and removing bias from models**: Keep bias features in data during training, remove what was learned from these bias features after training.